

CMR Specialist

Comprehensive Manual: Internal Mechanics & Full Capabilities (v2.0)

This manual explains **how the system works inside** - every pipeline stage, every enforcement rule, every state transition - and **everything it can do**, with the operator console guide, API reference, configuration, operations, and troubleshooting. It describes the system as deployed: model-proposed planning, OpenRouter inference, sealed tool module over a mock CMR database.

Reading guide: Sections 1-2 give the mental model. Sections 3-6 open each subsystem. Sections 7-10 are the operator references (UI, API, config, operations).

1. Governing Principles

- **Store-first (spec 05).** Every input is written to the database before any model is consulted. A crash mid-reasoning loses no user data; the case, input, and run rows already exist.
- **Model proposes, software enforces.** The model contributes judgment (intent, names, plan order, drafts). The harness contributes guarantees (allowlists, budgets, pattern checks, approvals, idempotency, audit). Neither side can do the other's job.
- **Patterned IDs are never trusted from the model.** RRR, NIN, phone, email, plate, chassis must pass deterministic patterns. If the model hallucinates digits, the value is rejected and the regex finding stands.
- **Names belong to the model.** Buyer/seller/requester names have no pattern; only the model can extract them. They are stored as *pending* (needing corroboration), never *valid*.
- **Fallbacks are visible, never silent.** When the model is unreachable or unparseable, deterministic templates cover - and every cover is flagged (*plan_fallback*, log lines). Silent degradation once hid a real bug; it cannot anymore.
- **Honest failure over invention.** With no evidence, the system says so (bounded fallback). With missing facts, it asks (prompts). With danger, it stops (approval gate).

2. System Architecture

Five layers, one direction of dependence. Requests flow down; nothing below is reachable except through the layer above.

Layer	Responsibility	Lives in
Ingress (web API)	Auth, guard, case/message/task/knowledge endpoints, health, docs	harness/api/main.py
Agent (intake)	Intent, extraction, validation, planning-at-creation	harness/agent/*.py
Agent (execution)	Worker lease loop, policy gate, step runner, drafting	harness/worker/, service.execute_task
Tool module (sealed)	CHECK/WRITE formulations; ONLY holder of DB access	harness/tools/module.py, mockdb.py
Store	Cases, fields, tasks, steps, approvals, audit, knowledge	harness/store/ (12 tables)

2.1 Request lifecycle

POST /cases - guard check - shell case row - input row - intent classification (model-first, rules veto) - extraction (model + regex merge) - branch: KNOWLEDGE_QUERY answers immediately; AUTOMATION_TASK creates a task **with its plan already stored** (queued if complete, waiting_for_input if facts are missing); CASE_UPDATE attaches and tries to resume waiting tasks; UNSUPPORTED refuses with a fixed message.

2.2 The two tool systems (do not confuse them)

- **Internal tools** (registry: `validate_fields`, `knowledge_lookup`, `payment_status_check`, `draft_solution`, ...): pure functions over the case store. They read and compute. They cannot touch the CMR database and leave no audit rows.
- **Tool-module formulations** (`CHECK.* / WRITE.*`): the only path to the (mock) CMR database. `CHECK` verifies and is free; `WRITE` mutates and always requires human approval plus writes an append-only audit row. The harness calls them through a thin client; the model and harness hold no DB credentials.

3. Intake Mechanics (guard - intent - extract - validate)

3.1 Guard

Runs before anything is stored. Blocks disallowed content (abuse, injection, out-of-scope sensitive requests) with a reason. Blocked text never reaches the store or the model.

3.2 Intent (model-first with veto)

The model classifies each input as `KNOWLEDGE_QUERY`, `AUTOMATION_TASK`, `CASE_UPDATE`, `NEEDS_CLARIFICATION`, or `UNSUPPORTED`. Deterministic veto rules override the model on safety-relevant patterns (e.g. explicit task verbs the model downplays, or unsafe requests it accepts). Low-confidence classifications degrade to `NEEDS_CLARIFICATION` rather than guessing.

3.3 Extraction (model proposes, regex disposes)

Two extractors run and merge. The regex floor captures patterned IDs (RRR 12 digits, NIN 11 digits non-phone-like, phones, emails, plates, chassis, dates, explicit buyer:/seller:/account: labels) and strips filler words between label and value ('plate number is ABC123XY' yields ABC123XY). The model proposes the full field map, which contributes what patterns cannot: buyer/seller/requester names and paraphrased facts. Merge rules: the model may ADD name fields freely; for patterned IDs its values must pass the same patterns or are discarded; conflicts keep the regex value. Model silence keeps the regex findings and logs *[extractor] model silent*.

3.4 Task typing and validation

`AUTOMATION_TASK` inputs are typed by keyword (`_infer_task_type`: RRR/Remita/payment - payment_reconciliation; ownership/buyer+seller - change_of_ownership; else general_support). Each type declares required fields (payment: RRR + date + account; ownership: plate + buyer + seller). `validate_for_task` compares against the case's merged field state; anything missing produces a waiting task plus a per-field prompt ('What is the vehicle plate number?').

4. Planning Mechanics (the model holds the pen)

4.1 propose_plan

At task creation, the model is shown the task type, known fields, and the ALLOWED tool list for that type, and asked to reply with only JSON: an ordered subset of tools, verifications before writes. The harness then enforces: unknown names are dropped (a test once slipped in 'rm_rf_everything'; it was discarded), the list is capped at `MAX_PLAN_STEPS` (12), `WRITE`-bearing steps are force-marked `requires_approval` regardless of the model's opinion, and every step gets a deterministic idempotency key. An empty, silent, or garbage proposal falls back to the static template - logged as *[planner] proposal failed* and flagged *plan_fallback: true*.

4.2 Plan-at-creation and stored-plan execution

The resulting plan is serialized into `task.plan_json` immediately - so even blocked tasks display their intended path. When the worker later runs the task it loads the `STORED` plan (revalidating policy) instead of inventing a new one; only if no plan exists does it propose fresh. Plan and execution can therefore never silently diverge. Family plans (multi-formulation `CHECK/WRITE` sequences for DND, confirmations, etc.) follow the same envelope with formulation IDs as steps.

4.3 Visibility flags

- **plan_proposed_by**: *model* (reasoned) or *template* (fallback cover) - present on intake responses, task payloads, and the console badge.
- **plan_fallback: true** means work continues without model reasoning. Occasional flags are normal; clusters mean quota/throttling - check the server log, not the code.

5. Execution Mechanics (worker, policy, idempotency)

5.1 Worker loop

The worker polls for queued tasks, leases one (status running, leased_by worker id), loads case fields, loads the stored plan, and runs a policy check. Heartbeats and queue age are exposed via /health (oldest_queued_job_age_s tells you instantly if the worker is down: queued tasks with a growing age).

5.2 Policy gate

Two rules. (1) Approval: any step flagged requires_approval, or a task created with approval_required != never, moves the task to waiting_approval and records an approval request - the worker will not proceed until a human approves (one approval unlocks the task; the gate never loops). (2) Tool confinement: steps may only name registered tools or catalogued formulations; anything else fails the task with an error_ref instead of running.

5.3 Step runner and idempotency

Each step runs in order with its declared arguments; args, result, and status are appended to run_steps. Formulation steps (CHECK:*/WRITE.*) route through the thin client to the tool module; knowledge_lookup attaches retrieved spans as citations; everything else runs in the internal registry. Before running a step, the worker checks its idempotency key against completed steps - replays and crash-restarts skip already-done work instead of duplicating it. A crashed task returns to the queue via lease expiry up to max_attempts (3), then fails with its error_ref preserved.

5.4 Follow-ups and resume

POST /cases/{id}/messages runs the same intake on the existing case: new facts merge into case state (field versions increment; history is kept), and _resume_waiting_tasks revalidates every waiting task - complete ones flip to queued automatically. The console's Card 2 calls this endpoint; the activity log reports resumed=N.

6. Tool Module and Knowledge Base

6.1 Tool module (the sealed executor)

The module exposes a CATALOG of formulations (8 CHECK + 7 WRITE in the current build) plus FAMILY_PLANS that bundle them into procedures. Execution is a five-stage pipeline: validate arguments - check policy/token - fire against the mock CMR DB (with injectable faults for testing) - record the append-only ToolAudit row - return a fixed-shape envelope the harness relays untouched. A six-exam conformance suite gates any mock-to-real cutover; nothing reaches a production registry without passing it.

6.2 Knowledge base (procedure memory)

Upload (Card 4 or POST /knowledge/documents) stores the file and chunks its text; Publish flips a version to *active*. Retrieval searches ACTIVE versions only - uploading without publishing is the most common cause of empty answers, by design (unreviewed text must not ground answers). Answers are drafted from cited spans and pass a validator: no spans means the bounded fallback ('I couldn't find supporting evidence in the published knowledge base...'), even when the model could guess. The mock DB is fact memory; the knowledge base is procedure memory - starve either and the corresponding capability degrades honestly.

7. Data Model and Configuration

7.1 Tables (12)

Table	Holds	Why it matters
cases / case_inputs	Case shells + every raw input	Nothing is ever lost; reruns are reproducible
extracted_fields	Field name/value/confidence/validation/version	Merge history; pending vs valid distinction
tasks / run_steps	Task state + plan_json; per-step args/result/status	Full execution trail; idempotent replay
approvals	Requests + human decisions	Human-in-the-loop proof

Table	Holds	Why it matters
tool_audit	Every formulation fire (append-only)	Proves what touched the CMR DB (0 rows = nothing did)
knowledge_*	Documents, versions, chunks, citations	What grounds answers; what is published
agent_runs / case_events	Model runs; case timeline	Latency, prompt versions, audit narrative

7.2 Configuration (.env, never committed)

Key	Purpose	Notes
INFERENCE_PROVIDER	openrouter (or gemini)	Selects gateway branch
OPENROUTER_API_URL/MODEL	Endpoint + model id	Needs model + Bearer + Referer/Title headers
INFERENCE_API_KEY	Model billing key	Per-day cost on flash-class models is cents
HARNESS_API_KEY	API auth (X-API-Key)	Default dev value; rotate for shared hosts
TOOL_MODULE_TOKEN	Module call token	Compose + worker must agree
DATABASE_URL	Store location	Defaults to local SQLite; Postgres for deploy
LLM_MIN_INTERVAL_S/CACHE_TL_S	Pacer + cache (2s / 600s)	Survives free-tier quotas; never blocks, only paces

Config does NOT auto-load .env: start web with `--env-file .env` and export the variables for the worker. Forgetting this is the most common local-deploy fault.

8. Full Capability Inventory

8.1 Task families

- **payment_reconciliation** (RRR + date + account): validates facts, retrieves procedure, verifies the receipt against the ledger (verified:true + last-4), drafts a summary. Straight-through when all facts are present.
- **change_of_ownership** (plate + buyer + seller): model extracts the names, regex validates the plate, plan executes validate - lookup - draft. Missing facts wait with prompts; follow-ups resume.
- **general_support**: knowledge-grounded answer with cited spans, or the bounded fallback.
- **Knowledge Q&A**: any procedure question answered only from published spans.
- **Approvals-gated work**: any task created with approval sensitive/always, or any plan containing WRITE steps, pauses for a human decision and records it.

8.2 Task lifecycle states

State	Meaning	How to move it
queued	Complete facts; awaiting worker	Automatic (seconds)
waiting_for_input	Missing facts; plan already visible	Answer prompts via follow-up
waiting_approval	Sensitive/WRITE steps need a human	Approval card: approve/reject
running / completed / failed	Executing / done / error kept	Poll; error_ref explains failures

9. Operator Console (<http://localhost:8080>)

Card	Action	Read the result
1 - New case	Paste question/task, Submit	intent, task_id, status, missing + prompts, plan flags
2 - Follow-up	Case ID auto-carries; paste fact, Attach	Log shows resumed=N; waiting tasks wake

Card	Action	Read the result
3 - Queue task	Type + approval level, Queue	Direct task; approval:always demos the gate
4 - Knowledge	Upload, then Publish version id	Chunk count; publish or answers stay empty
Case state	Refresh case / Refresh task	Fields, plan, plan:model/template(fallback) badge
Approval	Approve / Reject + reason	Gated tasks proceed or stop, recorded

9.1 Guided exercises

A - wait then resume. Card 1: *Musa sold his car to Adaeze Okafor, please transfer the ownership* (expect waiting_for_input, missing plate). Card 2: *the plate number is ABC123XY* (expect resume - completed).

B - straight-through payment. Card 1: *Please reconcile my Remita payment RRR 123456789012 made yesterday for account acct-X* (expect queued - completed, receipt verified paid, cited spans).

C - approval gate. Card 3 with approval:always, payment_reconciliation, *confirm receipt 123456789012* (expect waiting_approval; clear it in the Approval card).

D - unhappy path. Card 1 with a wrong RRR such as *000000000000* (expect completed-with-not-found verdict, never an invented match).

9.2 API essentials

Method + path	Purpose	Notes
POST /cases	Create case + intake	Body: text (+title/sender)
POST /cases/{id}/messages	Follow-up on a case	Merges facts; resumes waiting tasks
POST /cases/{id}/tasks	Queue task directly	Type + instructions + approval level
GET /cases/{id} - GET /tasks/{tid}	State + task detail	Task carries plan, flags, steps, approvals
POST /tasks/{tid}/approvals	Human decision	approved/rejected + reason
POST /knowledge/documents - .../publish	Upload + publish	Both steps required
GET /health - /docs	Liveness + API docs	Watch oldest_queued_job_age_s

9.3 Operations

```
# start (from project root; .env required)

nohup .venv/bin/python -m uvicorn harness.api.main:app \
--host 0.0.0.0 --port 8080 --env-file .env > /tmp/cmr_server.log 2>&1 &

nohup env $(grep -v '^#' .env | grep '=' | xargs) \
.venv/bin/python -m harness.worker.worker > /tmp/cmr_worker.log 2>&1 &

# health / stop

curl -s http://127.0.0.1:8080/health

kill -f harness.api.main; kill -f harness.worker.worker
```

Slow first starts (up to ~2 min) are normal on a loaded machine - imports are heavy. A curl that returns empty right after launch usually just raced startup; retry. If the port is busy, a stale process is squatting it: find with *pgrep -f harness* and kill by PID before relaunching.

10. Verification and Troubleshooting

10.1 Test suite (35 checks, must stay green)

File	Proves	Count
test_lean_spec.py	Spec lifecycle: intake, waiting, approvals, idempotency	7

File	Proves	Count
test_tool_module.py	Catalog, pipeline, audit, faults	13
test_llm_gemini.py	Provider wiring, cache, veto, merge	12
test_model_plans.py	Model ordering; invented tools dropped; silent-model fallback	3

10.2 Failure signatures

- Stuck *queued* + growing queue age: worker down - restart; verify heartbeat.
- Perpetual *waiting_for_input*: Refresh case names the missing fact - answer it in Card 2.
- *plan: template (fallback)* badges clustering: model throttled - server log shows *[planner]/[extractor]* lines; wait and retry, nothing is broken.
- Knowledge answers empty: version uploaded but not published - publish it.
- Empty curl after launch: raced startup - retry; persistent refusal: stale process on :8080.
- Buyer/seller values with extra words: extraction prompt guards this; report the exact text so the prompt can be hardened.